

EMBEDDED SYSTEMS & INTERNET OF THINGS (IoT)

TASK – 1

Introduction to Embedded Systems & IoT Device Design

Name: Deeksha Pandit

Branch: Electronics and Communication Engineering

Year: 4th Year

College: Lakshmi Narain College of Technology, Bhopal

1. INTRODUCTION

1.1 What is an Embedded System?

An embedded system is a computer-based electronic system designed to perform a specific function or a particular set of functions within a larger system.

Unlike general-purpose computers, embedded systems are developed for dedicated applications. They combine hardware and software to monitor inputs, process information, and produce the required output.

A typical embedded system consists of a microcontroller or microprocessor, memory, input devices such as sensors, and output devices such as LEDs, displays, motors, or buzzers.

Basic operation: Input → Processing → Output

For example, in the proposed temperature monitoring system:

Temperature Sensor → Arduino UNO → Temperature Processing → LED/Buzzer Alert

2. REAL-WORLD APPLICATIONS OF EMBEDDED SYSTEMS

2.1 Smart Home

Embedded systems are used in smart lights, automatic fans, security systems, smart thermostats, door locks, and home automation devices.

2.2 Automotive Systems

Modern vehicles contain embedded systems for engine control, airbag systems, anti-lock braking systems, parking assistance, infotainment, and vehicle monitoring.

2.3 Healthcare Devices

Embedded systems are used in digital thermometers, patient-monitoring equipment, heart-rate monitors, blood-pressure monitors, and other medical instruments.

2.4 Robotics

Robots use embedded controllers to process information received from sensors and control motors and other components.

2.5 Consumer Electronics

Washing machines, microwave ovens, televisions, printers, cameras, and many other electronic products use embedded systems to perform dedicated functions.

3. MICROCONTROLLER VS MICROPROCESSOR

A microcontroller is an integrated circuit that contains a processing unit along with memory and input/output peripherals. It is commonly used for dedicated control applications.

A microprocessor mainly contains the processing unit and generally requires external memory and peripherals to create a complete computing system.

Feature	Microcontroller	Microprocessor
Purpose	Dedicated control applications	General-purpose computing
Memory	Usually integrated	Generally external

I/O peripherals	Usually integrated	Often external
Power consumption	Generally low	Generally higher
Cost	Generally lower for control applications	Generally higher
Applications	IoT, robotics, automation, sensors	Computers and high-performance systems
Examples	Arduino-based systems, ESP32	Computer processors

4. COMPONENT STUDY

4.1 Arduino

Arduino is a popular microcontroller development platform used for embedded systems, robotics, automation, and IoT prototyping. In this project, Arduino UNO R3 is used as the main controller to read the temperature sensor, process the value, compare it with the threshold, control the LED/buzzer, and display status through the Serial Monitor.

4.2 ESP32

ESP32 is a microcontroller platform widely used for IoT applications. It provides Wi-Fi and Bluetooth along with GPIO and other interfaces, making it suitable for smart-home systems, wireless sensor networks, remote monitoring, and cloud-connected IoT devices.

4.3 Raspberry Pi Pico

Raspberry Pi Pico is a microcontroller development board based on the RP2040 microcontroller. It provides GPIO pins and interfaces for sensors, displays, LEDs, motors, and other components.

5. BASIC SENSORS

5.1 Temperature Sensor

A temperature sensor measures the temperature of its surrounding environment and produces an electrical signal corresponding to the measured temperature. A TMP36 is used in this project and provides an analog voltage related to temperature.

5.2 Motion Sensor

A motion sensor detects movement in its surrounding environment. A PIR sensor is a common example and is used in security systems, automatic lighting, and smart-home applications.

5.3 Light Sensor

A light sensor measures light intensity. An LDR is a common light sensor whose resistance changes according to the amount of light falling on it.

6. MINI PROJECT – SMART TEMPERATURE MONITORING AND ALERT SYSTEM

6.1 Project Overview

The mini project developed for this task is a Smart Temperature Monitoring and Alert System. The system continuously monitors temperature using a TMP36 temperature sensor connected to an Arduino UNO. When the temperature is within the normal range, the system displays the temperature and indicates normal

operation. When the temperature reaches or crosses the predefined threshold of 30°C, the system activates an LED and buzzer to provide an alert. The project is implemented using simulation, as the task allows a simulator.

6.2 Objective

1. To understand the basic working of an embedded system.
2. To interface a temperature sensor with a microcontroller.
3. To read and process analog sensor data.
4. To monitor temperature continuously.
5. To generate an alert when the temperature crosses a predefined threshold.
6. To gain practical experience with Arduino and sensor-based IoT prototyping.

6.3 Components Required

S. No.	Component	Quantity
1	Arduino UNO R3	1
2	TMP36 Temperature Sensor	1
3	Breadboard	1
4	LED	1
5	220Ω Resistor	1
6	Piezo Buzzer	1
7	Jumper Wires	As required

6.4 Circuit Connections

TMP36 Temperature Sensor

TMP36 Pin	Connection
Left pin	Arduino 5V
Middle pin	Arduino A0
Right pin	Arduino GND

LED

LED Connection	Arduino
Positive pin through 220Ω resistor	Digital Pin 8
Negative pin	GND

Buzzer

Buzzer Connection	Arduino
Positive pin	Digital Pin 9
Negative pin	GND

6.5 Circuit Diagram

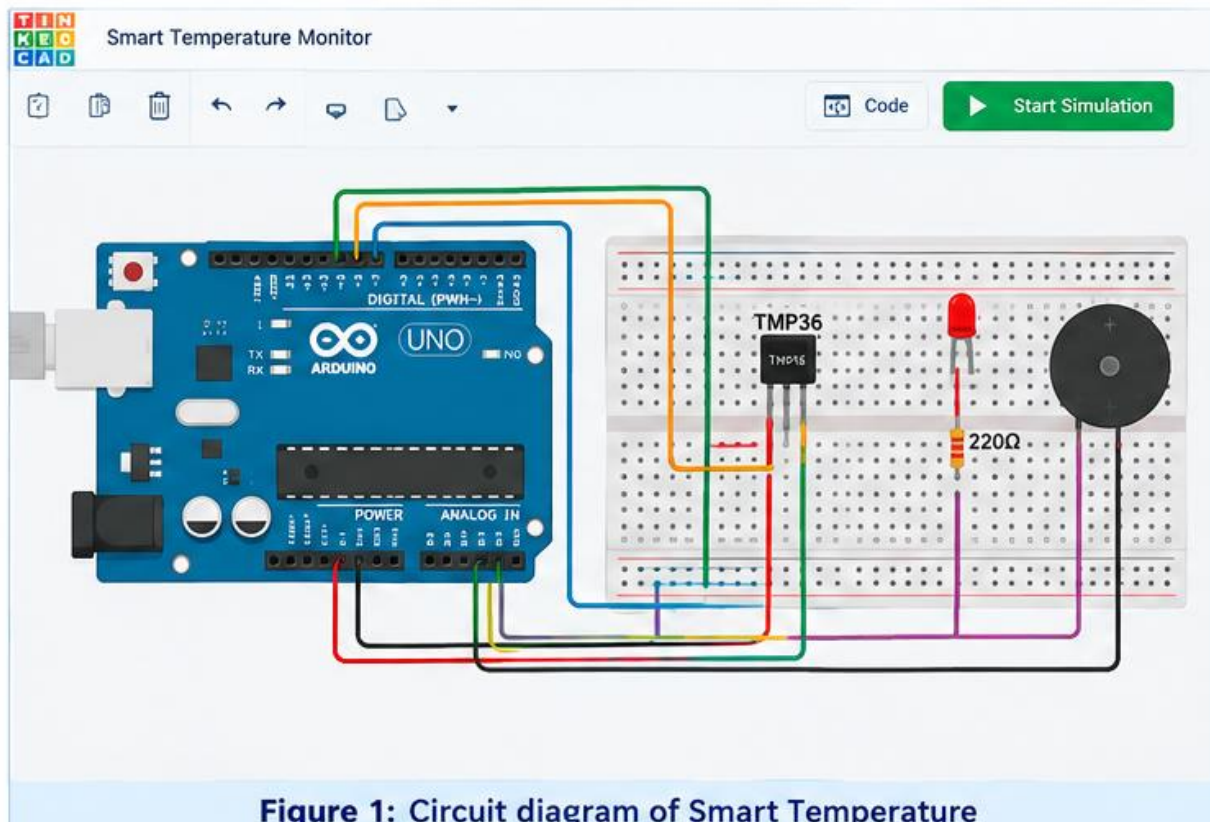


Figure 1: Circuit diagram of Smart Temperature

Figure 1: Circuit diagram of the Smart Temperature Monitoring and Alert System

6.6 Working Principle

Step 1 – Temperature Sensing: The TMP36 detects the surrounding temperature and produces an analog voltage.

Step 2 – Analog Reading: The Arduino UNO receives the sensor output through analog pin A0.

Step 3 – Voltage Conversion: The Arduino converts the analog reading into a voltage value.

Step 4 – Temperature Calculation: The voltage is converted into temperature in degrees Celsius.

Step 5 – Threshold Comparison: The calculated temperature is compared with the predefined threshold of 30°C.

Step 6 – Output: Below 30°C, the LED and buzzer remain OFF and normal status is displayed. At or above 30°C, the LED and buzzer are activated and a high-temperature alert is displayed.

Overall process:

TMP36 → Arduino A0 → Temperature Calculation → Threshold Comparison → LED/Buzzer Alert

6.7 Arduino/C++ Code

```
const int tempPin = A0;
const int ledPin = 8;
const int buzzerPin = 9;
```

```

const float threshold = 30.0;

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buzzerPin, OUTPUT);

  Serial.begin(9600);

  Serial.println("Smart Temperature Monitoring System");
  Serial.println("-----");
}

void loop() {

  int sensorValue = analogRead(tempPin);

  float voltage = sensorValue * (5.0 / 1023.0);

  float temperature = (voltage - 0.5) * 100.0;

  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" °C");

  if (temperature >= threshold) {

    digitalWrite(ledPin, HIGH);
    digitalWrite(buzzerPin, HIGH);

    Serial.println("ALERT: Temperature is HIGH!");

  }
  else {

    digitalWrite(ledPin, LOW);
    digitalWrite(buzzerPin, LOW);

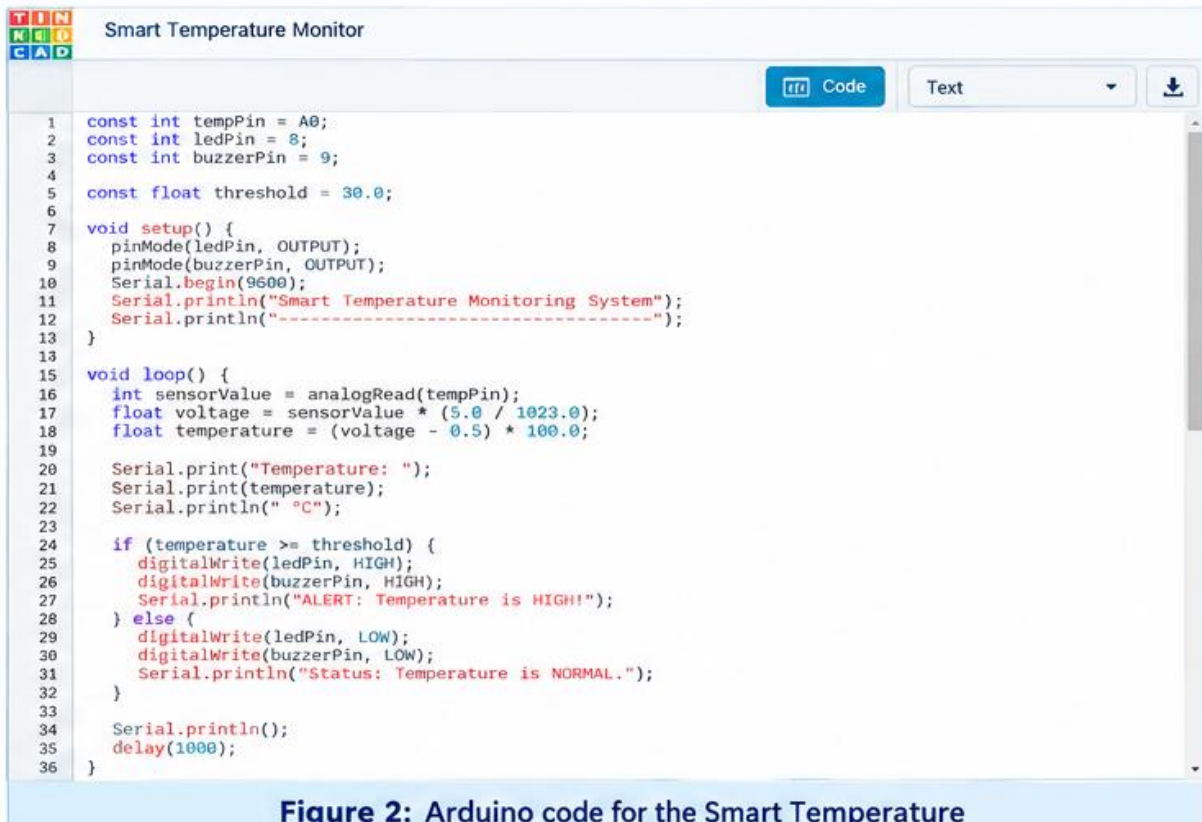
    Serial.println("Status: Temperature is NORMAL.");
  }

  Serial.println();

  delay(1000);
}

```

6.8 Code Screenshot



```
1 const int tempPin = A0;
2 const int ledPin = 8;
3 const int buzzerPin = 9;
4
5 const float threshold = 30.0;
6
7 void setup() {
8   pinMode(ledPin, OUTPUT);
9   pinMode(buzzerPin, OUTPUT);
10  Serial.begin(9600);
11  Serial.println("Smart Temperature Monitoring System");
12  Serial.println("-----");
13 }
14
15 void loop() {
16   int sensorValue = analogRead(tempPin);
17   float voltage = sensorValue * (5.0 / 1023.0);
18   float temperature = (voltage - 0.5) * 100.0;
19
20   Serial.print("Temperature: ");
21   Serial.print(temperature);
22   Serial.println(" °C");
23
24   if (temperature >= threshold) {
25     digitalWrite(ledPin, HIGH);
26     digitalWrite(buzzerPin, HIGH);
27     Serial.println("ALERT: Temperature is HIGH!");
28   } else {
29     digitalWrite(ledPin, LOW);
30     digitalWrite(buzzerPin, LOW);
31     Serial.println("Status: Temperature is NORMAL.");
32   }
33
34   Serial.println();
35   delay(1000);
36 }
```

Figure 2: Arduino code for the Smart Temperature

Figure 2: Arduino code for the Smart Temperature Monitoring and Alert System

6.9 Code Explanation

Sensor and Output Pins:

```
const int tempPin = A0;
```

```
const int ledPin = 8;
```

```
const int buzzerPin = 9;
```

These define the Arduino pins used by the temperature sensor, LED, and buzzer.

Temperature Threshold:

```
const float threshold = 30.0;
```

The threshold is set to 30°C.

Reading the Sensor:

```
int sensorValue = analogRead(tempPin);
```

The Arduino reads the analog signal from the TMP36.

Voltage Calculation:

```
float voltage = sensorValue * (5.0 / 1023.0);
```

The analog reading is converted into voltage.

Temperature Calculation:

```
float temperature = (voltage - 0.5) * 100.0;
```

The sensor voltage is converted into temperature in Celsius.

Alert Condition:

if (temperature >= threshold)

The Arduino checks whether the temperature has reached or crossed the threshold.

High Temperature:

`digitalWrite(ledPin, HIGH);`

`digitalWrite(buzzerPin, HIGH);`

The LED and buzzer are activated when the temperature is high.

Normal Temperature:

`digitalWrite(ledPin, LOW);`

`digitalWrite(buzzerPin, LOW);`

The LED and buzzer remain OFF when the temperature is below the threshold.

6.10 Simulation Result – Normal Condition

When the temperature is below 30°C, the system considers the temperature normal.

Example:

Temperature: 25.10 °C

Status: Temperature is NORMAL.

The LED and buzzer remain OFF.

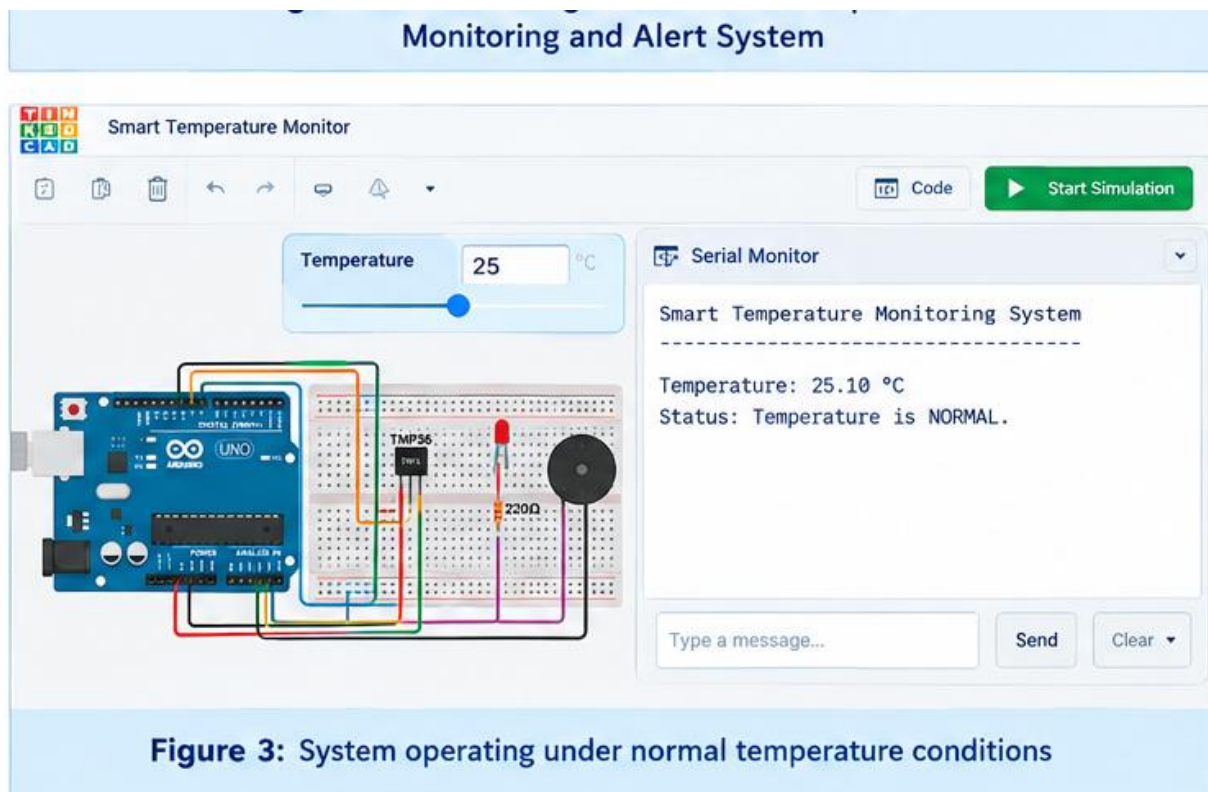


Figure 3: System operating under normal temperature conditions

6.11 Simulation Result – High Temperature Condition

When the temperature reaches or exceeds 30°C, the system generates an alert.

Example:

Temperature: 31.20 °C

ALERT: Temperature is HIGH!

The LED and buzzer are activated.

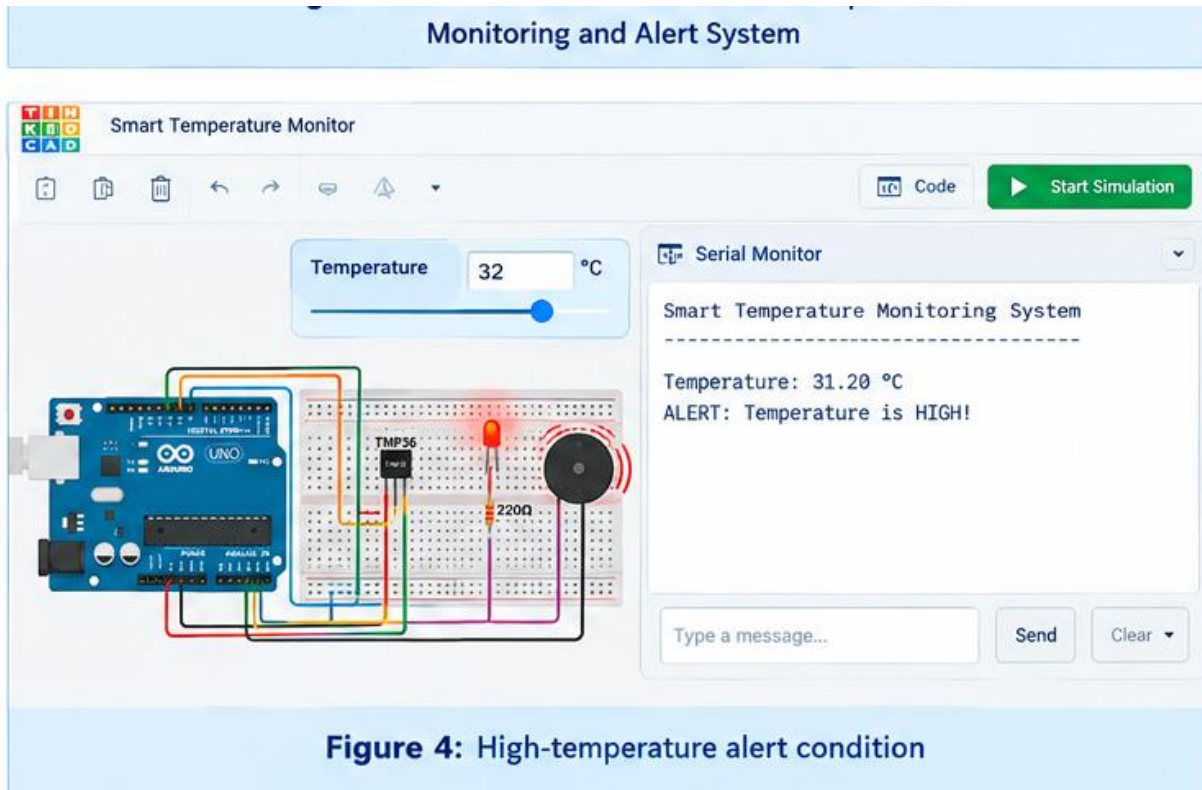


Figure 4: High-temperature alert condition

Figure 4: High-temperature alert condition

7. REAL-WORLD APPLICATIONS OF THE PROJECT

7.1 Smart Home Temperature Monitoring

The system can monitor room temperature and provide an alert when the temperature becomes too high.

7.2 Server Room Monitoring

Temperature sensors can monitor server rooms and provide warnings when equipment may be exposed to excessive heat.

7.3 Industrial Equipment Monitoring

Similar systems can monitor the temperature of machines and equipment to help identify overheating conditions.

7.4 Greenhouse Monitoring

Temperature monitoring can be used in greenhouse environments to help maintain suitable conditions for plants.

7.5 IoT-Based Remote Monitoring

By replacing the Arduino UNO with ESP32, temperature information could be sent to a cloud platform or mobile application for remote monitoring.

8. ADVANTAGES

1. Simple and easy to understand.
2. Low-cost components can be used.
3. Provides continuous temperature monitoring.
4. Provides both visual and audible alerts.
5. Easy to modify and expand.
6. Can be implemented using simulation without physical hardware.
7. Can be extended into a complete IoT-based monitoring system.

9. FUTURE SCOPE

The project can be enhanced by adding:

- LCD or OLED display.
- ESP32 for Wi-Fi connectivity.
- Cloud data storage.
- Mobile notifications.
- Temperature history and graphs.
- Automatic cooling-fan control.
- Multiple temperature sensors.
- Web or mobile dashboard.

10. CONCLUSION

The Smart Temperature Monitoring and Alert System demonstrates the basic principles of embedded systems and IoT device design. The TMP36 sensor provides temperature information to the Arduino UNO, which processes the signal, calculates temperature, and compares it with a predefined threshold.

When the temperature is normal, the system indicates normal operation. When the temperature reaches or exceeds 30°C, the LED and buzzer provide an alert.

The project demonstrates sensor interfacing, analog input, microcontroller programming, threshold-based decision making, and embedded-system simulation. It can be extended into a connected IoT application using a Wi-Fi-enabled controller such as ESP32.

11. REFERENCES

1. NPTEL – Introduction to Embedded Systems
2. Arduino – Getting Started Guide
3. Autodesk Tinkercad Circuits

4. Wokwi – Arduino and ESP32 Simulator

5. Embedded Systems and IoT learning resources